

Evaluación Parcial N°3

Encargo con Presentación

Estudiante

Sigla	Nombre Asignatura	Tiempo Asignado	% Ponderación
SCY1101	Programación para la Ciencia de Datos	4 semanas	40%

1.Instrucciones generales

Descripción

- Esta evaluación busca que los estudiantes vivan una experiencia integral de desarrollo y despliegue de proyectos de análisis de datos en un contexto profesional.
- El desafío es construir una solución end-to-end que integre al menos tres fuentes de datos diferentes (por ejemplo: CSV/Excel, API REST y una base de datos SQL o NoSQL), desarrolle un pipeline ETL automatizado, visualice resultados mediante dashboards interactivos y gestione todo el ciclo de vida del proyecto con prácticas profesionales de colaboración (Git) y despliegue (Docker).
- Cada estudiante defenderá de manera individual el funcionamiento de la solución, la arquitectura técnica, la lógica de visualización y los procesos colaborativos aplicados, simulando una defensa de proyecto en un entorno laboral real.
- La distribución de los porcentajes en esta evaluación es la siguiente:

Evaluación	Tipo de situación evaluativa	Distribución de porcentajes en el ET	Individual/Grupal
Evaluación Sumativa III)/ Evaluación Parcial N°3	Encargo	10%	Grupal
	Presentación	30%	Individual
Total		40%	

- El **tiempo** asignado para desarrollar esta evaluación en **el laboratorio** es de **4 semanas para el encargo 15 minutos** para la presentación y se realiza de manera **(individual, parejas o equipos)**.

Instrucciones específicas de la Evaluación:**Ítem I: Consideraciones para el encargo:**

Focos de observación

- Organización y reproducibilidad del proyecto y sus componentes.
- Robustez del pipeline ETL y la integración de fuentes.
- Calidad y profundidad de la documentación técnica.
- Buenas prácticas de colaboración en Git y despliegue en Docker.
- Capacidad de evidenciar testing, manejo de errores y automatización.

El informe y entregable deben incluir:

- Pipeline ETL completo (scripts y notebooks): Debe integrar al menos tres fuentes de datos, incluir transformaciones robustas, validación de esquemas y manejo avanzado de errores.
- Documentación técnica: README claro, diagramas de arquitectura, documentación de APIs, manual de usuario y guía de despliegue.
- Dashboard interactivo: Código fuente y acceso a demo (Plotly Dash o Streamlit), con visualizaciones diferenciadas por audiencia.
- Código colaborativo en repositorio Git: Historial de ramas, merges y evidencias de trabajo colaborativo (uso de pull requests, revisión de código, issues).
- Containerización con Docker: Dockerfiles, docker-compose, variables de entorno para configuración externa; scripts de despliegue.

Estructura de carpetas recomendada:

- /etl/: Scripts y notebooks del pipeline de integración y transformación.
- /dashboards/: Código de dashboards, archivos de configuración, capturas.
- /docs/: Manuales, guías, diagramas, referencias.
- /api/: Código de la API RESTful.
- /docker/: Dockerfiles, docker-compose, scripts de entorno.
- /tests/: Scripts y reportes de testing automatizado.
- /data/: Datos originales y transformados.
- /repo/: Evidencia de uso profesional de Git.

Aspectos formales a evaluar:

- Código limpio, modular y bien documentado (docstrings).
- Testing automatizado e informes de pruebas.
- Configuración por variables de entorno y logging profesional.
- Documentación exhaustiva y actualizada.
- Optimización para ejecución eficiente y reproducible.

Materiales requeridos: PC/laptop, acceso a GitHub/GitLab, Docker, Python 3, librerías de ciencia de datos.

Ítem II: Consideraciones para la Presentación:

Focos de observación:

- Claridad y detalle en la defensa del pipeline ETL y arquitectura técnica.
- Justificación y análisis de decisiones de diseño, herramientas y procesos.
- Capacidad de adaptar el discurso y las visualizaciones para distintas audiencias (ejecutiva, técnica, operativa).
- Argumentación sobre prácticas colaborativas, uso de Git y estrategias de despliegue en Docker.
- Reflexión crítica sobre las lecciones aprendidas y las oportunidades de mejora.

La presentación debe incluir:

- Demo en vivo del pipeline end-to-end funcionando (de preferencia en entorno Docker).
- Explicación de la arquitectura y decisiones técnicas clave (diagramas, esquemas).
- Presentación y análisis de dashboards interactivos diferenciados por audiencia y valor de negocio.
- Evidencia y explicación de la colaboración profesional (ramas, merges, revisiones de código).
- Análisis de despliegue y uso de Docker, mostrando orquestación y optimización.
- Lecciones aprendidas y propuestas de mejora sobre el proceso técnico y colaborativo.

Aspectos formales:

- Slides claros y estructurados.
- Uso de notebook o dashboard interactivo.
- Lenguaje profesional y técnico en la exposición.
- Evidencia de material de apoyo (diagramas, capturas, links a repositorio/despliegue).

1. Pauta de Evaluación

Categoría	% logro	Descripción niveles de logro
Muy buen desempeño	100%	Demuestra un desempeño destacado, evidenciando el logro de todos los aspectos evaluados en el indicador.
Buen desempeño	80%	Demuestra un alto desempeño del indicador, presentando pequeñas omisiones, dificultades y/o errores.
Desempeño aceptable	60%	Demuestra un desempeño competente, evidenciando el logro de los elementos básicos del indicador, pero con omisiones, dificultades o errores.
Desempeño incipiente	30%	Presenta importantes omisiones, dificultades o errores en el desempeño, que no permiten evidenciar los elementos básicos del logro del indicador, por lo que no puede ser considerado competente.
Desempeño no logrado	0%	Presenta ausencia o incorrecto desempeño.

Indicador de Evaluación	Categorías de Respuesta					Ponderación Indicador de Evaluación
	Muy buen desempeño 100%	Buen desempeño 80%	Desempeño aceptable 60%	Desempeño incipiente 30%	Desempeño no logrado 0%	
Dimensión: Encargo						
1. Pipeline ETL robusto, integra múltiples fuentes, validación de esquemas, manejo avanzado de errores, optimización para grandes volúmenes	Integra múltiples fuentes, validación robusta, manejo avanzado de errores, óptimo para grandes volúmenes	Integra principales fuentes, validación adecuada, manejo de errores básico	Integra fuentes principales pero validación o errores superficiales	Fuentes limitadas o manejo de errores deficiente	No integra correctamente	20%

2. Documenta completamente la arquitectura del sistema, APIs desarrolladas y procesos de instalación y configuración	Documenta arquitectura, APIs y procesos con ejemplos y diagramas claros, guía de instalación completa	Documenta adecuadamente pero omite detalles o ejemplos	Documentación básica pero funcional	Documentación incompleta o con errores	No documenta	20%
3. Dashboard interactivo con visualizaciones avanzadas, diferenciación por audiencia, terminología de negocio, rendimiento y accesibilidad óptimos	Dashboard avanzado, visualizaciones complejas y diferenciadas por audiencia, responsive, rendimiento óptimo	Dashboard funcional, algunas vistas diferenciadas, visualizaciones relevantes	Dashboard simple o sin diferenciación clara	Dashboard limitado o con errores	No desarrolla dashboard	25%
4. Uso profesional de Git, branching, pull requests, resolución de conflictos, historia de commits limpia, uso de herramientas colaborativas	Branching profesional, revisiones de código, commits claros, uso de issues y herramientas	Branching adecuado, revisiones básicas	Branching elemental, sin revisión profunda	Branching deficiente	No usa git correctamente	15%
5. Construye y despliega aplicación usando Docker con contenedores optimizados, docker-compose para orquestación y configuración externa	Contenedores optimizados, docker-compose, variables de entorno, documentación clara	Contenedores funcionales, docker-compose básico	Contenedores básicos	Contenedores con errores	No desarrolla containerización	20%
Dimensión: Presentación						
6. Demuestra el funcionamiento completo del pipeline end-to-end	Demo fluida, explicación detallada y	Demo funcional,	Demo simple, explicación básica	Demo incompleta	No demuestra funcionamiento	30%

explicando la arquitectura técnica y decisiones de diseño	respuestas sólidas	explicación adecuada				
7. Presenta dashboards interactivos explicando la adaptación para diferentes audiencias y el valor de negocio generado	Presenta dashboards adaptados, explica valor de negocio con ejemplos, evidencia comprensión de usuarios	Presenta dashboards con explicación adecuada	Presentación superficial	Presentación incompleta	No evidencia valor o no presenta dashboards	30%
8. Explica el proceso de desarrollo colaborativo, herramientas utilizadas y metodologías de gestión de proyecto aplicadas	Explica proceso colaborativo, herramientas y metodología; propone mejoras, análisis crítico	Explica colaboración con ejemplos adecuados	Explicación básica o superficial	Explicación incompleta o errónea	No explica o es incorrecto	40%
Total						100%